

포팅 메뉴얼

기술 스택 버전

- 백엔드
 - JAVA : OpenJDK 21 (LTS)
 - Spring Boot : 3.5.3
 - Gradle : 8.14.3
 - FastAPI : 0.114.3
 - Node.js : 18.4.2
- 프론트엔드
 - Node.js : 20.19.4
 - Next.js : 15.4.4
 - pnpm : 10.13.1
 - TypeScript : 5.8.3
 - TailwindCSS : 4.1.11
- 인프라
 - Jenkins : 2.516.1
 - Docker Engine : 28.3.2
 - Docker Compose : 2.38.2
 - MySQL : 8.0.43
 - Redis : 7.2.10
- IDE
 - VisualStudio Code : 1.102.3
 - IntelliJ IDEA : 2025.1.3

포트번호

포트	프로토콜	프로세스/컨테이너	용도 추정
80	TCP	<code>docker-proxy</code> → <code>nginx_proxy</code>	NGINX HTTP
443	TCP	<code>docker-proxy</code> → <code>nginx_proxy</code>	NGINX HTTPS
8080	TCP	<code>docker-proxy</code> → <code>nearzoom-backend</code>	Spring Boot
3306	TCP	<code>docker-proxy</code> → <code>nearzoom-mysql</code>	MySQL
6379	TCP	<code>docker-proxy</code> → <code>nearzoom-redis</code>	Redis
22	TCP	<code>sshd</code>	SSH
53	TCP/UDP	<code>systemd-resolve</code>	로컬 DNS 스텝
68	UDP	<code>systemd-networkd</code>	DHCP 클라이언트
323	UDP	<code>chronyd</code>	NTP 동기화
8989	TCP	<code>httpd</code> (Apache)	별도 웹/프록시
8988	TCP	<code>java</code> (gerrit.service)	Gerrit HTTP
29418	TCP	<code>java</code> (gerrit.service)	Gerrit SSH 기본 포트
9090	TCP	<code>java</code> (jenkins.service)	Jenkins
8000	TCP	<code>python</code>	FASTAPI
3000	TCP/UDP	<code>javascript</code>	Node

환경 변수

- DB

```

PROD_MYSQL_PORT = 3306
PROD_MYSQL_DB = nearzoom
PROD_MYSQL_USERNAME = ssafy
PROD_MYSQL_PASSWORD = ssafy
HOST = mysql

```

- Google 소셜 로그인

```

OAUTH_GOOGLE_CLIENT_ID = 401822101331-33jm32r1v4ivnsorri8bs8
3lg03r76i5.apps.googleusercontent.com

```

```
OAUTH_GOOGLE_CLIENT_SECRET = GOCSPX-aAYpZXeRzOd9Ak3GJp4pOxJLE75
OAUTH_GOOGLE_REDIRECT_URI = https://api.nearzoom.store/login/oauth2/code/google
```

- Kakao 소셜 로그인

```
OAUTH_KAKAO_CLIENT_ID = 56adcf231c323161cbaab03d6c8e9013
OAUTH_KAKAO_CLIENT_SECRET = xiGjlo9hUDXArfewNQeO1TsPxukT54WN
OAUTH_KAKAO_REDIRECT_URI = https://api.nearzoom.store/login/oauth2/code/kakao
```

- JWT

```
JWT_SECRET = askdfjklasjfkhdjksafhjasdhfkjlsadjkfhsad
ACCESSTOKEN_EXPIRE = 900000
REFRESHTOKEN_EXPIRE = 86400000
JWT_REFRESH_TOKEN_TTL = 86400000
```

- Redis

```
REDIS_HOST = nearzoom-redis
REDIS_PORT = 6379
```

- Redirect URL

```
REDIRECT_URL = http://localhost:3000
```

- Image Server

```
IMAGE_SERVER_URL = https://image.nearzoom.store
```

- LiveKit

```
LIVEKIT_API_KEY = APIFDvuyF72kTrQ
LIVEKIT_API_SECRET = 1eleYyi4H127DRfeQLp6bezgzEI596mrSCsHQOz
```

it89F

LIVEKIT_URL = wss://common-sw1q8xgz.livekit.cloud

- yml
 - docker-compose.yml

```
version: '3.8'

services:
  redis:
    image: redis:7.2
    container_name: nearzoom-redis
    ports:
      - "6379:6379"
    restart: always
    volumes:
      - /var/lib/jenkins/volumes/nearzoom/redis-data:/data

  mysql:
    image: mysql:8.0
    container_name: nearzoom-mysql
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: ${PROD_MYSQL_PASSWORD}
      MYSQL_DATABASE: ${PROD_MYSQL_DB}
      MYSQL_USER: ${PROD_MYSQL_USERNAME}
      MYSQL_PASSWORD: ${PROD_MYSQL_PASSWORD}
    ports:
      - "3306:3306"
    volumes:
      - ./mysql-data:/var/lib/mysql

  spring:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: nearzoom-backend
    depends_on:
```

```
- mysql
- redis
ports:
  - "8080:8080"
environment:
  SPRING_DATASOURCE_URL: jdbc:mysql://${HOST}:${PROD_MY
SQL_PORT}/${PROD_MYSQL_DB}?useSSL=false&useUnicode=true
&allowPublicKeyRetrieval=true
  PROD_MYSQL_USERNAME: ${PROD_MYSQL_USERNAME}
  PROD_MYSQL_PASSWORD: ${PROD_MYSQL_PASSWORD}
  REDIS_HOST: ${REDIS_HOST}
  REDIS_PORT: ${REDIS_PORT}
  REDIS_PASSWORD: ${REDIS_PASSWORD}
  OAUTH_GOOGLE_CLIENT_ID: ${OAUTH_GOOGLE_CLIENT_ID}
  OAUTH_GOOGLE_CLIENT_SECRET: ${OAUTH_GOOGLE_CLIENT
_SECRET}
  OAUTH_GOOGLE_REDIRECT_URI: ${OAUTH_GOOGLE_REDIRECT
_URI}
  OAUTH_KAKAO_CLIENT_ID: ${OAUTH_KAKAO_CLIENT_ID}
  OAUTH_KAKAO_CLIENT_SECRET: ${OAUTH_KAKAO_CLIENT_SE
CRET}
  OAUTH_KAKAO_REDIRECT_URI: ${OAUTH_KAKAO_REDIRECT_U
RI}
  SPRING_JWT_SECRET: ${JWT_SECRET}
  SPRING_JWT_ACCESS_EXPIRATION: ${ACCESSTOKEN_EXPIR
E}
  SPRING_JWT_REFRESH_EXPIRATION: ${REFRESHTOKEN_EXPI
RE}
  SPRING_JWT_REFRESH_TOKEN_TTL: ${JWT_REFRESH_TOKEN_
TTL}
  APP_REDIRECT_URL: ${REDIRECT_URL}
  IMAGE_SERVER_URL: ${IMAGE_SERVER_URL}
  LIVEKIT_API_KEY: ${LIVEKIT_API_KEY}
  LIVEKIT_API_SECRET: ${LIVEKIT_API_SECRET}
  LIVEKIT_URL: ${LIVEKIT_URL}
restart: always

networks:
```

```
default:
  external: true
  name: nearzoom_default
```

- application.yml

```
spring:
  config:
    import: "optional:file:..env[.properties]"
  profiles:
    default: prod
```

- application-prod.yml

```
spring:
  datasource:
    url: jdbc:mysql://${HOST}:${PROD_MYSQL_PORT}/${PROD_MYSQL_DB}?useSSL=false&characterEncoding=UTF-8&serverTimezone=UTC&useUnicode=true&allowPublicKeyRetrieval=true
    driver-class-name: com.mysql.cj.jdbc.Driver
    username: ${PROD_MYSQL_USERNAME}
    password: ${PROD_MYSQL_PASSWORD}
  servlet:
    multipart:
      max-file-size: 1000MB
      max-request-size: 1000MB

  jpa:
    hibernate:
      ddl-auto: validate
    show-sql: true
    properties:
      hibernate:
        format_sql: true
        dialect: org.hibernate.dialect.MySQL8Dialect

  data:
    redis:
```

```
host: ${REDIS_HOST}
port: ${REDIS_PORT}
password: ${REDIS_PASSWORD}
```

security:

oauth2:

client:

registration:

google:

client-id: \${OAUTH_GOOGLE_CLIENT_ID}

client-secret: \${OAUTH_GOOGLE_CLIENT_SECRET}

redirect-uri: \${OAUTH_GOOGLE_REDIRECT_URI}

scope:

- email

- profile

kakao:

client-id: \${OAUTH_KAKAO_CLIENT_ID}

client-secret: \${OAUTH_KAKAO_CLIENT_SECRET}

client-authentication-method: client_secret_post

authorization-grant-type: authorization_code

redirect-uri: \${OAUTH_KAKAO_REDIRECT_URI}

scope:

- profile_nickname

- profile_image

- account_email

client-name: Kakao

provider:

kakao:

authorization-uri: https://kauth.kakao.com/oauth/authorize

token-uri: https://kauth.kakao.com/oauth/token

user-info-uri: https://kapi.kakao.com/v2/user/me

user-name-attribute: id

jwt:

secret: \${JWT_SECRET}

access-expiration: \${ACCESSTOKEN_EXPIRE}

```
refresh-expiration: ${REFRESHTOKEN_EXPIRE}  
refresh-token-ttl: ${JWT_REFRESH_TOKEN_TTL}
```

app:

```
redirect-url: ${REDIRECT_URL}
```

mybatis:

```
mapper-locations: classpath:/mappers/**/*.xml
```

```
type-aliases-package: com.ssafy.nearzoom.domain.myroom.entity
```

configuration:

```
map-underscore-to-camel-case: true
```

image-server:

```
url: ${IMAGE_SERVER_URL}
```

webclient:

```
max-memory-size: 10485760
```

springdoc:

api-docs:

```
path: /api-docs
```

swagger-ui:

```
path: /swagger-ui.html
```

```
tags-sorter: alpha
```

```
operations-sorter: method
```

```
packages-to-scan: com.ssafy.nearzoom
```

```
default-consumes-media-type: application/json
```

```
default-produces-media-type: application/json
```

livekit:

api:

```
key: ${LIVEKIT_API_KEY}
```

```
secret: ${LIVEKIT_API_SECRET}
```

```
url: ${LIVEKIT_URL}
```

- JenkinsFile

```

pipeline {
    agent any

    environment {
        ENV_FILE_ID = '720df111-d8d6-4035-b618-1e03ae9ec433'
        GIT_CREDENTIALS_ID = 'd719d5d7-c8aa-4e55-940d-c2963d52219e'
        GIT_REPO_URL = 'https://lab.ssafy.com/s13-webmobile1-sub1/S13P11A605.git'
        GIT_BRANCH = 'backend'
    }

    stages {
        stage('Clone') {
            steps {
                git credentialsId: "${GIT_CREDENTIALS_ID}", url: "${GIT_REPO_URL}", branch: "${GIT_BRANCH}"
            }
        }

        stage('Setup Secrets') {
            steps {
                withCredentials([file(credentialsId: "${ENV_FILE_ID}", variable: 'ENV_FILE')]) {
                    sh '''
                        cd backend
                        cp "$ENV_FILE" .env
                        chmod 644 .env
                    '''
                }
            }
        }

        stage('Build') {
            steps {
                sh '''
                    cd backend
                    chmod +x gradlew
                '''
            }
        }
    }
}

```

```

        ./gradlew clean build -x test --no-daemon
        cp build/libs/nearzoom-0.0.1-SNAPSHOT.jar app.jar
    """
}
}

stage('Docker Compose Up') {
    steps {
        sh """
            cd backend
            docker compose down || true
            docker compose up -d --build
            docker compose ps
        """
    }
}
}
}
}

```

- DockerFile

```
FROM openjdk:21-jdk-slim
```

```
COPY app.jar app.jar
```

```
ENTRYPOINT ["java", "-Dspring.profiles.active=prod,secret", "-jar",
"app.jar"]
```

데이터베이스 접속 방법

- MySQL 설정

Host: mysql

DB: nearzoom

User: ssafy

Password: ssafy

Port: 3306

```
Charset: utf8mb4
Collation: utf8mb4_unicode_ci
```

- Redis 설정

```
Host: nearzoom-redis
Port: 6379
Database: 0
```

- 데이터베이스 초기화
 - JPA와 MyBatis를 함께 사용하고 JPA의 ddl-auto: validate로 되어있기 때문에 직접 테이블을 생성하거나 변경해주어야 합니다.

외부 서비스 정보

소셜 인증 (OAuth2)

- Google
 - **Redirect URI:** <https://api.nearzoom.store/login/oauth2/code/google>
 - **Scope:** email, profile
- Kakao
 - **Redirect URI:** <https://api.nearzoom.store/login/oauth2/code/kakao>
 - **Scope:** profile_nickname, profile_image, account_email
- AI
 - GPT-Image-1 api <https://platform.openai.com/docs/api-reference/images/createEdit>
- LiveKit Cloud

자동 빌드 및 배포

EC2 세팅

```
# Java 21 설치
sudo apt update
```

```
sudo apt install openjdk-21-jdk -y
```

Docker 설치

```
# 패키지 인덱스 업데이트
```

```
sudo apt-get update
```

```
# 필요 패키지 설치
```

```
sudo apt-get install \  
  apt-transport-https \  
  ca-certificates \  
  curl \  
  software-properties-common
```

```
# Docker의 공식 GPG 키 추가
```

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key ad  
d -
```

```
# Docker 저장소 추가
```

```
sudo add-apt-repository \  
  "deb [arch=amd64] https://download.docker.com/linux/ubuntu \  
  $(lsb_release -cs) \  
  stable"
```

```
# 패키지 인덱스 다시 업데이트
```

```
sudo apt-get update
```

```
# Docker CE 설치
```

```
sudo apt-get install docker-ce docker-ce-cli containerd.io
```

```
# Docker 서비스 시작 및 자동 시작 설정
```

```
sudo systemctl start docker  
sudo systemctl enable docker
```

```
# 현재 사용자를 docker 그룹에 추가
```

```
sudo usermod -aG docker ${USER}
```

Jenkins 설치

```
# 1) Jenkins GPG 키/레포 등록
curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io.key | sudo tee /usr/share/keyrings/jenkins-keyring.asc > /dev/null
echo 'deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] https://pkg.jenkins.io/debian-stable binary/' \
| sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null

# 2) 패키지 인덱스 갱신 & 설치
sudo apt-get update
sudo apt-get install -y jenkins

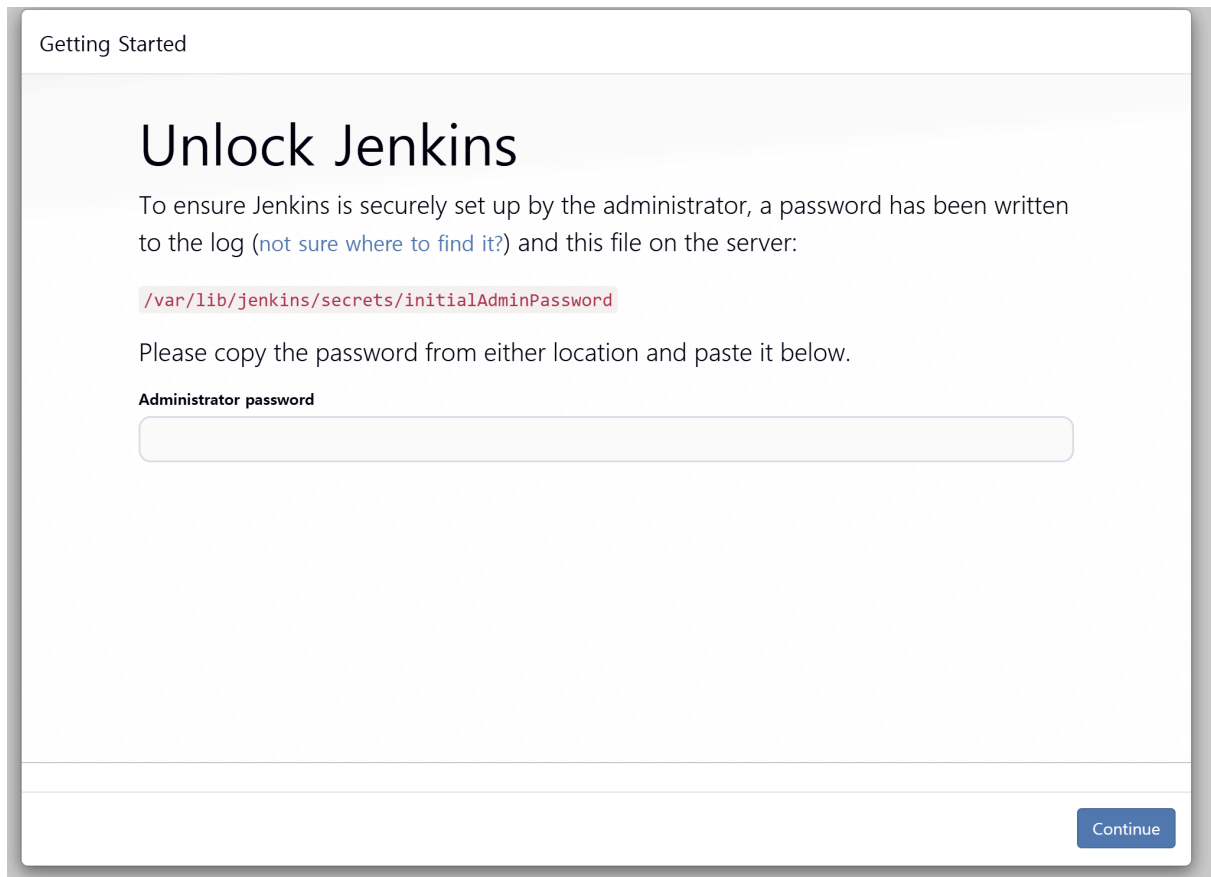
# 3) 서비스 등록/기동
sudo systemctl enable --now jenkins
```

Jenkins 포트 번호 변경

```
# 설정 파일 열기
sudo vi /lib/systemd/system/jenkins.service
# 8080을 9090으로 변경
sudo systemctl daemon-reload
sudo systemctl restart jenkins
```

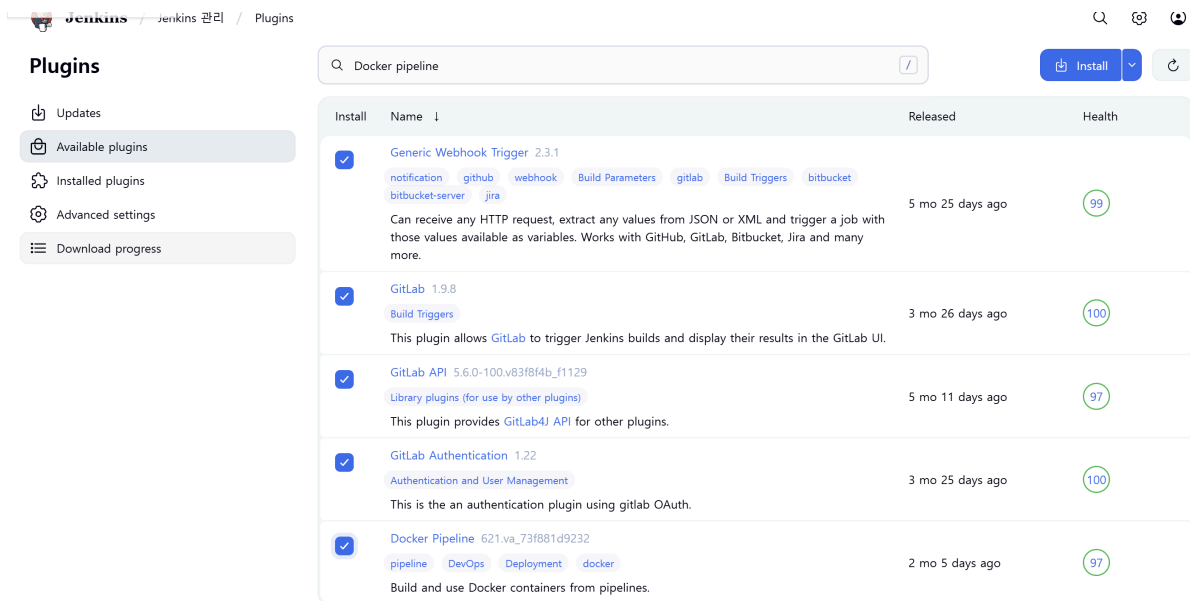
Jenkins 설정

```
# 초기 관리자 비밀번호 확인
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```



확인한 비밀번호를 입력

플러그인 설치



GitLab AccessToken 생성

Token description

Expiration date

2025-09-03

Select a role

Maintainer

Select scopes

Scopes set the permission levels granted to the token. [Learn more.](#)

- ☒ **api**
Grants complete read and write access to the scoped project API, including the container registry, the dependency proxy, and the package registry.
- ☒ **read_api**
Grants read access to the scoped project API, including the Package Registry.
- ☐ **create_runner**
Grants create access to the runners.
- ☐ **manage_runner**
Grants access to manage the runners.
- ☐ **k8s_proxy**
Grants permission to perform Kubernetes API calls using the agent for Kubernetes.
- ☐ **self_rotate**
Grants permission for token to rotate itself.
- ☒ **read_repository**
Grants read access (pull) to the repository.
- ☐ **write_repository**
Grants read and write access (pull and push) to the repository.
- ☐ **ai_features**
Grants access to GitLab Duo related API endpoints.

[Create project access token](#) [Cancel](#)

생성하고 발급되는 AccessToken 저장

Jenkins에 GitLab 연결

GitLab connections

Connection name ?
A name for the connection

potato

GitLab host URL ?
The complete URL to the GitLab server (e.g. http://gitlab.mydomain.com)

https://lab.ssafy.com/

Credentials ?
API Token for accessing GitLab

GitLab API token (gitlab at)

[+ Add](#)

[고급](#)

[Test Connection](#)

[Save](#) [Apply](#)

Jenkins Credentials Provider: Jenkins

Add Credentials

Domain

Global credentials (unrestricted) ▼

Kind

GitLab API token ▼

Scope ?

Global (Jenkins, nodes, items, all child items, etc) ▼

API token

.....

ID ?

Description ?

gitlab at

아까 저장한 AccessToken을 활

WebHook 설정

on

이 빌드는 매개변수가 있습니다 ?

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

☐ Build after other projects are built ?

☐ Build periodically ?

☒ Build when a change is pushed to GitLab. GitLab webhook URL: `http://3.37.174.35:9090/project/peach` ?

Enabled GitLab triggers

☒ Push Events ?

☐ Push Events in case of branch delete ?

☒ Opened Merge Request Events ?

☐ Build only if new commits were pushed to Merge Request ?

☐ Accepted Merge Request Events ?

☐ Closed Merge Request Events ?

Rebuild open Merge Requests ?

Never

Save Apply

사진속 URL 저장

Secret token ?

1bb41210ac7aba43780386d51eb09804

Generate

Secret token 생성

ration

Pipeline script from SCM

SCM ?

Git

Repositories ?

Repository URL ?

https://lab.ssafy.com/kwon2615/test_back_huni.git

Credentials ?

dnlwlgns1117/*****

+ Add

고급

Add Repository

Jenkins Credentials Provider: Jenkins

Kind

Username with password

Scope ?

Global (Jenkins, nodes, items, all child items, etc)

Username ?

dnlwlgns1117

☐ Treat username as secret ?

Password ?

.....

ID ?

Description (optional)

URL

http://3.37.174.35:9090/project/peach

⚠ The secret token is cleared on save unless it is updated.

The URL must be percent-encoded if it contains one or more special characters.

+ Add URL masking ?

Custom headers </> 0

No custom headers configured.

Secret token

.....

Used to validate received payloads. Sent with the request in the X-GitLab-Token HTTP header.

Trigger

☒ Push events

☐ All branches

☐ Wildcard pattern

☒ Regular expression

^master\$

Regular expressions such as ^(feature|hotfix)/ are supported.

GitLab에서 WebHook 설정하기

env파일 Jenkins Credential에 숨기기

gitlab credential 설정

숨기려는 파일 업로드

Jenkins 관리 -> Credentials -> Stores scoped to Jenkins의 (global) -> Add Credential

Secret file 선택 후 업로드

개발자가 GitLab에 코드를 push하거나 MR을 만들면 GitLab 웹훅이 EC2의 Jenkins를 호출하고, Jenkins는 저장소를 체크아웃한 뒤 리포지토리에 있는 Jenkinsfile을 실행합니다. Jenkinsfile은 먼저 Dockerfile로 Spring Boot 이미지를 빌드·태깅하고, 이어서 docker-compose.yml을 사용해 배포하며, 새 백엔드 이미지를 적용해 Spring Boot 컨테

이너를 교체합니다. 만약 구성이 바뀌지 않으면 MySQL/Redis는 기존 볼륨을 유지한 채 그대로 둡니다.

```
pipeline {
  agent any

  environment {
    ENV_FILE_ID = '720df111-d8d6-4035-b618-1e03ae9ec433'
    GIT_CREDENTIALS_ID = 'd719d5d7-c8aa-4e55-940d-c2963d52219e'
    GIT_REPO_URL = 'https://lab.ssafy.com/s13-webmobile1-sub1/S13P11A
605.git'
    GIT_BRANCH = 'backend'
  }

  stages {
    stage('Clone') {
      steps {
        git credentialsId: "${GIT_CREDENTIALS_ID}", url: "${GIT_REPO_U
RL}", branch: "${GIT_BRANCH}"
      }
    }

    stage('Setup Secrets') {
      steps {
        withCredentials([file(credentialsId: "${ENV_FILE_ID}", variable: 'E
NV_FILE')]) {
          sh '''
            cd backend
            cp "$ENV_FILE" .env
            chmod 644 .env
          '''
        }
      }
    }

    stage('Build') {
      steps {
        sh '''
```

```

        cd backend
        chmod +x gradlew
        ./gradlew clean build -x test --no-daemon
        cp build/libs/nearzoom-0.0.1-SNAPSHOT.jar app.jar
    """
}
}

stage('Docker Compose Up') {
    steps {
        sh """
            cd backend
            docker compose down || true
            docker compose up -d --build
            docker compose ps
        """
    }
}
}
}

```

FROM openjdk:21-jdk-slim

COPY app.jar app.jar

ENTRYPOINT ["java", "-Dspring.profiles.active=prod,secret", "-jar", "app.jar"]